5 – Temporal Classification Stacks

From Single-Date RF Maps to Time-Series Classification Cubes

Chris Reudenbach

Inhaltsverzeichnis

Goals

- Use the existing Burgwald data pipeline as a **fixed base**.
- Run and understand two scripts:
  - `bfast_kndvi_gdalcubes.R` (BFAST kNDVI on NetCDF cube)
  - `04_classification_cdse_one_date.R` (single-date CDSE classification)
- Create and use **training polygons** for supervised classification.

- Generalise the **single-date RF classification** to multiple time points.
- Build a **temporal classification stack** (multi-date RF maps in one cube).
- Prepare the basis for later **stratification** and **similarity mapping**.

Prerequisites

Before starting this worksheet you must have completed:

- Worksheet **4** – **Execute the Burgwald Data Pipeline**
- A working project setup with:
  - monthly Sentinel-2 NetCDF cube (e.g. `burgwald_2018_2022_all.nc`)
  - at least **one** CDSE-based predictor stack (e.g. `pred_stack_2018.tif`)
  - AOI, DEM, CLC, DWD, OSM as in WS-04

If this is not the case: finish WS-04 first.

Task 0 – Create training polygons

Before any classification tasks, you must manually create a **training polygon layer**.

Requirements:

- At least **two classes** (e.g. `clearcut`, `forest`, `other`).
- Saved as **GeoPackage**: `data/processed/train_areas_burgwald.gpkg`.
- Geometry: **POLYGON** / **MULTIPOLYGON**.
- CRS identical to your CDSE predictor stacks.
- Field name for labels: **`class`**.

Steps:

1. Open QGIS.
2. Load Burgwald base layers (RGB, NDVI, DEM, etc.).
3. Digitise polygons for the classes you want to separate.
4. Save to:

```
data/processed/train_areas_burgwald.gpkg
```

5. Verify structure in R:

```
x <- sf::st_read("data/processed/train_areas_burgwald.gpkg")
names(x)
```

6. If `class` is missing, add it in QGIS and assign class labels.

Task 1 – Run the BFAST kNDVI script (as is)

Script 1: `bfast_kndvi_gdalcubes.R` ("BFAST-based kNDVI change detection on gdalcubes NetCDF")

1. Place (or reference) the script in your project (e.g. `src/04_bfast_kndvi_gdalcubes.R`).
2. Make sure `root_folder <- here::here()` is available (from `01_setup-burgwald.R`).
3. Run the script **unchanged**.

Expected input (adapt if your filenames differ):

- `data/burgwald_2018_2022_all.nc`

Expected output:

- `data/burgwald_bfast_results.nc`

Minimal checks:

- Confirm that `burgwald_bfast_results.nc` exists.
- Optionally open it, e.g.:

```
bfast_star <- stars::read_stars("data/burgwald_bfast_results.nc")
bfast_star
```

Task 2 – Run the single-date CDSE classification (as is)

Script 2: `04_classification_cdse_one_date.R` ("Perform classification for ONE CDSE-based predictor stack")

Input assumptions:

- `data/pred_stack_2018.tif` (single-date predictor stack)
- `data/processed/train_areas_burgwald.gpkg` (training polygons with `class` field)

1. Check the paths in the script and adjust **only if absolutely necessary**:
    - `pred_stack_file`
    - `train_poly_file`
2. Run the script unchanged.

Expected outputs (example names from the script):

- `data/classification_kmeans_2018.tif`
- `data/classification_mlc_2018.tif`
- `data/classification_rf_2018.tif`
- `data/rf_model_2018_cdse.rds`
- `data/rf_confusion_2018_cdse.rds`

Task 3 – Identify the RF classification core

In `04_classification_cdse_one_date.R`:

1. Locate the **Random Forest** training block (`caret::train(...)`):
    - Which object holds the trained RF model?
    - On which columns (`predictor_cols`) is it trained?
2. Locate the **RF map prediction**:
    - the line that calls `terra::predict(pred_stack, rf_model, ...)`
    - the line that writes the RF map to disk (`classification_rf_2018.tif`)

Write down (in your notes or as comments) 2–3 bullet points:

- Input(s) needed for RF classification.
- Output(s) produced.
- Fixed assumptions (e.g. `class` field, training file path, CRS).

Task 4 – Prepare multiple CDSE predictor stacks

We treat the single-date classification as a **template**.

1. Decide on at least **3–4 time slices** (years or dates) for which you want RF classifications, for example:
   - `pred_stack_2018.tif`
   - `pred_stack_2019.tif`
   - `pred_stack_2020.tif`
   - `pred_stack_2021.tif`
2. Ensure that all these files exist in your `data/` folder. If necessary, adapt your CDSE pipeline so that it exports additional years/dates using the same structure as `pred_stack_2018.tif`.
3. Use the **same training polygons** (`train_areas_burgwald.gpkg`) for all dates.

Task 5 – Turn the RF classification into a function

Create a new script, e.g. `04_classification_cdse_multi_date.R`, and:

1. Copy the relevant RF parts from `04_classification_cdse_one_date.R`:
   - loading of `pred_stack_file`
   - loading of training polygons
   - extraction of training data (`exactextractr`)
   - RF training (`caret::train`)
   - RF prediction (`terra::predict`)
   - writing the RF classification raster
2. Wrap the RF logic into a function, e.g.:

```
classify_one_date_rf <- function(pred_stack_file,
                                 train_poly_file,
                                 class_field,
                                 out_raster_file) {
  # 1) load raster + training polygons
  # 2) extract training data
  # 3) fit RF model
  # 4) predict RF map
```

```
  # 5) write RF map to out_raster_file
  # 6) optionally return model + confusion matrix
}
```

3. Test the function for **one** date first (e.g. 2018) and confirm that:
   - the RF output raster is created,
   - a confusion matrix is computed,
   - the model is stored if you need it.

Task 6 – Loop over multiple dates and build a classification stack

Extend `04_classification_cdse_multi_date.R`:

1. Define a vector of predictor stack paths, e.g.:

```
years <- 2018:2021
pred_files <- here::here("data", paste0("pred_stack_", years, ".tif"))
```

2. For each `pred_stack_file` in `pred_files`:
   - derive an output name such as `classification_rf_<year>.tif`,
   - call your `classify_one_date_rf()` function.

3. After the loop, build a **multi-layer classification stack**:

```
rf_files <- here::here("data", paste0("classification_rf_", years, ".tif"))
rf_stack <- terra::rast(rf_files)

terra::writeRaster(
  rf_stack,
  here::here("data", "classification_rf_timeseries.tif"),
  overwrite = TRUE
)
```

(If you prefer NetCDF, you can export the stack accordingly.)

Expected result:

- a raster stack where each layer = RF classification for one time slice.

Task 7 – Link to change detection (optional, short)

Optional, for those who also want to align classification with BFAST results:

1. Load the BFAST result file `burgwald_bfast_results.nc` as a `stars` or `terra` object.
2. Co-register (if necessary) the RF classification stack and the BFAST layers.
3. Inspect:
    - regions with **strong change magnitude** (from BFAST),
    - and their **temporal RF class behaviour** (from your classification stack).

This step is exploratory only; formal stratification comes later.

Take-home: temporal stacks as basis for stratification

Write **one short paragraph** (max. 6–7 sentences):

- Why is it useful to have **RF classification maps for multiple time points** in a single stack?
- How could such a stack be used to:
    - define **temporal strata** (e.g. "always stable forest", "recent clearcuts", "gradual change"),
    - find **areas of similar behaviour** (spatial clusters with similar class trajectories)?

This will be the conceptual basis for the next step: **stratified sampling and similarity mapping based on temporal class trajectories.**